



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии» (ИУ7)

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Название: Обработка разреженных матриц

Дисциплина: Типы и структуры данных

Студент

ИУ7-31Б

(Группа)

Постнов С.А

(Фамилия И. О.)

Преподаватель

Силантьева А.В.

Оглавление

Цель работы	3
Условие задачи	3
Описание ТЗ.....	3
1. Описание входных данных	3
2. Описание результатов работы программы.....	4
3. Способ обращения к программе	4
4. Задача программы	4
5. Описание возможных аварийных ситуаций и ошибок пользователя....	5
Описание внутренних структур данных	5
Тестовые случаи	6
Сравнение эффективности алгоритмов при различной размерности и заполненности.....	6
Выводы	8
Ответы на контрольные вопросы	9

Цель работы

Реализация алгоритмов обработки разреженных матриц, сравнение этих алгоритмов со стандартными при различной размерности и степени разреженности.

Условие задачи

Разреженная (содержащая много нулей) матрица хранится в форме 3-х объектов:

- вектор A содержит значения ненулевых элементов;
- вектор JA содержит номера столбцов для элементов вектора A;
- связный список IA, в элементе Nk которого находится номер компонент в A и JA, с которых начинается описание строки Nk матрицы A

- 1) Смоделировать операцию сложения двух матриц, хранящихся в этой форме, с получением результата в той же форме.
- 2) Произвести операцию сложения, применяя стандартный алгоритм работы с матрицами.
- 3) Сравнить время выполнения операций и объем памяти при использовании этих 2-х алгоритмов при различном проценте заполнения матриц.

Описание ТЗ

1. Описание входных данных

На вход программе из файлов поступают целочисленные матрицы, которые хранятся в нем в «простом» виде, т.е.:

2 2 // Кол-во строк и столбцов через пробел
5 -7 // Элементы
0 13 // матрицы

Ограничения:

1. Элементы матрицы – целые числа
2. Любой код, не лежащий в диапазоне от 0 до 5 включительно, считается ошибочным

2. Описание результатов работы программы

В результате работы программы введенные матрицы будут представлены в виде обычного двумерного массива и 3-ех векторов. После этого выведется меню программы, содержащее в себе следующие пункты:

- 0) Выход
- 1) Вывести матрицу
- 2) Заполнить пустую матрицу
- 3) Выполнить сложение матриц, применяя стандартный алгоритм работы, и вывести
- 4) Выполнить сложение матриц, хранящихся в виде трех векторов, и вывести
- 5) Сравнить время выполнения операций и объем памяти при использовании двух алгоритмов при различной заполненности

3. Способ обращения к программе

Обращение к программе происходит из командной строки путем запуска исполняемого файла «*app.exe*».

4. Задача программы

Задача данной программы заключала в себе несколько составных частей:

- Сложение двух матриц различными алгоритмами
- Измерение времени и памяти на сложение двух матриц различными алгоритмами

5. Описание возможных аварийных ситуаций и ошибок пользователя

- Некорректное имя файла:
 - Код возврата: 1
 - Сообщение: «Не удалось открыть файл. Попробуйте снова.»
- В файле (или при вводе вручную) некорректные данные:
 - Код возврата: 2
 - Сообщение: «Не удалось прочитать данные. Попробуйте снова.»
- Некорректная размерность в файле (или при вводе вручную):
 - Код возврата: 3
 - Сообщение: «Не удалось получить корректную размерность. Попробуйте снова.»
- Ошибка выделения памяти:
 - Код возврата: 4
 - Сообщение: «Не удалось выделить память. Попробуйте снова.»
- Некорректный код меню:
 - Код возврата: 5
 - Сообщение: «Некорректный код. Попробуйте снова.»

Описание внутренних структур данных

Тип данных, описывающий структуру для хранения матрицы в виде трех векторов (разреженный вид):

```
typedef struct
{
    int *a;    // Ненулевые элементы
    int *ja;   // Индексы столбцов ненулевых элементов
    int *ia;   // Индексы столбцов элементов, с которых начинается строка
    int count; // Кол-во ненулевых элементов матрицы
} spar_mtrx_t;
```

Тип данных, описывающий структуру для хранения результатов измерений:

```
typedef struct measure
{
    long long unsigned time; // Время выполнения
    int mem;                // Используемая память
    int fill;               // Процент заполнения
    int n;                  // Кол-во строк
    int m;                  // Кол-во столбцов
    char mtrx_type;         // Тип хранения
} measurement_table;
```

Тестовые случаи

Позитивные тесты:

1. Сложение матриц из одного элемента
2. Сложение противоположных друг другу матриц
3. Сложение ненулевой матрицы с нулевой
4. Сложение нулевых матриц

Негативные тесты:

1. Некорректный элемент матрицы
2. Некорректное имя файла
3. Некорректная размерность
4. Неверный код выбора пункта меню
5. Размеры матриц не равны
6. Кол-во элементов меньше введенной размерности

Сравнение эффективности алгоритмов при различной размерности и заполненности

Таблица замеров при различном проценте заполненности матриц и различной размерности (в микросекундах, по результатам 10 запусков каждого алгоритма с каждой размерностью и заполненностью):

TYPE	SIZE	FILL (%)	MEMORY (Bytes)	TIME (mcs)
ORDINARY	30	5	7680	2
SPARES	30	5	960	1
ORDINARY	50	5	20800	6
SPARES	50	5	2400	3
ORDINARY	70	5	40320	12
SPARES	70	5	4480	5

ORDINARY	100	5	81600	24
SPARES	100	5	8800	10
ORDINARY	30	8	7680	2
SPARES	30	8	1392	3
ORDINARY	50	8	20800	6
SPARES	50	8	3600	4
ORDINARY	70	8	40320	12
SPARES	70	8	6832	8
ORDINARY	100	8	81600	25
SPARES	100	8	13600	19
ORDINARY	30	10	7680	2
SPARES	30	10	1680	2
ORDINARY	50	10	20800	8
SPARES	50	10	4400	5
ORDINARY	70	10	40320	13
SPARES	70	10	8400	10
ORDINARY	100	10	81600	25
SPARES	100	10	16800	24
ORDINARY	30	12	7680	2
SPARES	30	12	1968	2
ORDINARY	50	12	20800	11
SPARES	50	12	5200	6
ORDINARY	70	12	40320	12
SPARES	70	12	9968	12
ORDINARY	100	12	81600	26
SPARES	100	12	20000	30
ORDINARY	30	15	7680	2
SPARES	30	15	2400	2
ORDINARY	50	15	20800	8
SPARES	50	15	6400	7
ORDINARY	70	15	40320	12
SPARES	70	15	12320	15
ORDINARY	100	15	81600	26
SPARES	100	15	24800	36
ORDINARY	30	20	7680	4
SPARES	30	20	3120	3
ORDINARY	50	20	20800	8
SPARES	50	20	8400	15
ORDINARY	70	20	40320	12
SPARES	70	20	16240	23
ORDINARY	100	20	81600	26
SPARES	100	20	32800	51
ORDINARY	30	30	7680	2
SPARES	30	30	4560	5
ORDINARY	50	30	20800	6
SPARES	50	30	12400	15
ORDINARY	70	30	40320	12
SPARES	70	30	24080	34
ORDINARY	100	30	81600	25
SPARES	100	30	48800	72
ORDINARY	30	50	7680	2
SPARES	30	50	7440	8
ORDINARY	50	50	20800	6
SPARES	50	50	20400	29
ORDINARY	70	50	40320	14
SPARES	70	50	39760	61
ORDINARY	100	50	81600	26
SPARES	100	50	80800	116

ORDINARY	30	90	7680	2
SPARES	30	90	13200	10
ORDINARY	50	90	20800	6
SPARES	50	90	36400	30
ORDINARY	70	90	40320	14
SPARES	70	90	71120	62
ORDINARY	100	90	81600	26
SPARES	100	90	144800	120
ORDINARY	30	100	7680	2
SPARES	30	100	14640	10
ORDINARY	50	100	20800	6
SPARES	50	100	40400	28
ORDINARY	70	100	40320	12
SPARES	70	100	78960	54
ORDINARY	100	100	81600	29
SPARES	100	100	160800	113

Исходя из результатов представленной выше таблице, можно сделать вывод, что при низкой заполненности матриц ($\leq 10\%$) для любой из замеренных размерностей сложение матриц, представленных в разреженном виде, осуществляется быстрее классического алгоритма. На промежутке от 10% до 13% алгоритмы обработки работают примерно одинаковое время. При возрастании процента заполненности ($>10\%$), можно наблюдать стабильное превосходство во времени классического сложения над разреженным.

Заметим, что выигрыш в памяти у разреженной матрицы наблюдается до 50% заполненности включительно, однако далее объем такой матрицы превосходит классическую.

Выводы

В ходе работы были изучены способы и алгоритмы работы с разреженными матрицами, а также проведены замерные эксперименты, по результатам которых можно сделать вывод, что хранение разреженных матриц в виде 3-х векторов с ненулевыми элементами выгодно, когда матрица имеет процент заполненности, находящейся в окрестности 10-13%. Таким образом получится снизить затраты на хранение и обработку данных. До 50% заполненности программисту необходимо самостоятельно выбрать,

какие на данный момент ресурсы необходимо экономить, так как обработка матриц в разреженном виде происходит дольше, но занимает меньше памяти. От 50% до 100% заполненности лучше всего подходит хранение и обработка матрицы в классическом виде.

Ответы на контрольные вопросы

- 1) Разреженная матрица – матрица, большая часть которой заполнена нулями. Можно хранить схемой Кнута, кольцевой схемой и в разреженном строчном(столбовом) формате
- 2) Под обычную матрицу выделяется $n * m * \text{sizeof}(\text{elem_type})$ байт памяти.

Под разреженную матрицу память выделяется по-разному для конкретного случая, который зависит от: 1) Кол-ва ненулевых элементов; 2) Размера матрицы; 3) Типа элементов

- 3) Происходит обработка только ненулевых элементов. Таким образом, количество операций будет пропорционально количеству ненулевых элементов
- 4) Разреженную матрицу эффективнее всего использовать в случае, когда заполненность матрицы довольно большая